Suppawat Rattanalaor (Gluy), https://github.com/Gluyspt/SYS-101-00020---Bare-Metal-Kernel

Level 1:

```
(base) gluyspt@Gluys-Macbook-Air moss-kernel-modified % just run

warning: `moss` (bin "moss") generated 8 warnings (run `cargo fix --bin "moss" -p moss` to apply 3 suggestions)
    Finished `release` profile [optimized] target(s) in 15.58s
warning: the following packages contain code that will be rejected by a future version of Rust: memchr v2.8.0
note: to see what the problems were, use the option `--future-incompat-report`, or run `cargo report future-incompatibilities --id 1`
     Running `/Users/gluyspt/moss-kernel-modified/scripts/qemu_runner.py target/aarch64-unknown-none-softfloat/release/moss --init /bin/ash`
[    0.000000] arch::arm64::boot::memory: Adding memory region from FDT Px40000000 (0x80000000 bytes)
[    0.000000] arch::arm64::boot::memory: Reserving kernel text Px40080000 - Px4119e000
[    0.000000] arch::arm64::boot::memory: Reserving FDT Px68000000 (0x100000 bytes)
[    0.000000] arch::arm64::boot::memory: Reserving initrd 0x48000000 - 0x68000000
[    0.000000] libkernel::memory::allocators::phys: Buddy allocator initialized. Managing 524288 pages, 379904 free.
[    0.000000] drivers::fs::dev: devfs initialized
[    0.000000] drivers::interrupts::arm_gic_v3: ARM GICv3 found: dist @ <0x8000000, 0x10000>, rdist @ <0x80a0000, 0xf60000> (stride=0x20000)
[    0.000000] drivers::interrupts::arm_gic_v3: GICv3: Redistributor for core 0 (MPIDR=0x80000000) is awake.
[    0.000000] drivers::interrupts::arm_gic_v3: GICv3: CPU interface for core 0 enabled.
[    0.000000] interrupts: Setting device intc@8000000 as the interrupt root
[    0.000000] interrupts: Device timer claimed interrupt: Ppi(14)
[    0.003674] interrupts: Device pl011@9000000 claimed interrupt: Spi(1)
[    0.009606] libkernel::sync::per_cpu: Setup 20 per_cpu objects.
[    0.011390] interrupts: Device CPU Messenger claimed interrupt: Ipi(0)
[    0.011786] arch::arm64::proc::vdso: VDSO mapped to: 0xffff810000000000 (0x1000 bytes)
[    0.032496] drivers::fs::proc: procfs initialized
[    0.033184] drivers::fs::sys: sysfs initialized
[    0.044524] drivers::interrupts::arm_gic_v3: GICv3: Redistributor for core 1 (MPIDR=0x80000001) is awake.
[    0.044914] drivers::interrupts::arm_gic_v3: GICv3: CPU interface for core 1 enabled.
[    0.045446] arch::arm64::boot::secondary: CPU 1 online.
[    0.045970] drivers::interrupts::arm_gic_v3: GICv3: Redistributor for core 2 (MPIDR=0x80000002) is awake.
[    0.046384] drivers::interrupts::arm_gic_v3: GICv3: CPU interface for core 2 enabled.
[    0.046740] arch::arm64::boot::secondary: CPU 2 online.
[    0.047300] drivers::interrupts::arm_gic_v3: GICv3: Redistributor for core 3 (MPIDR=0x80000003) is awake.
[    0.047656] drivers::interrupts::arm_gic_v3: GICv3: CPU interface for core 3 enabled.
[    0.048000] arch::arm64::boot::secondary: CPU 3 online.
/ #
```

Level 2:
Change from Earliest Eligible Virtual Deadline First scheduling (EEVDF) to round-robin scheduler.
Round robin scheduler (RR) works by giving each task a fixed time slice. If the time slice is up and that task is not finished yet, that task is then pushed to the back of the queue.

1. BTreeMap -> VecDeque
   VecDeque is more simple for queuing tasks.

   src/runqueue.rs line 72
   self.queue.push_back(old_task);
   Do the rotate task here. RR is simple, every time you preempt, the task gets sent to the back of the queue. There's no case where you preempt without rotating the task like in EEVDF.

2. Tick()
   EEVDF does update virtual run time, advance eligible time and compare it against the virtual deadline.
   RR just checks if the task used up all the fixed time slices or not.
   Elapsed became now - start (time used)
   Then compared to the DEFAULT_TIME_SLICE if it has used up all that.
   src/sched_task.rs line 65

3. find_next_runnable_desc()
   EEVDF does this by finding the best candidate from the run queue.
   RR just does it in a queue. First come first serve.

src/runqueue.rs line 114

4. Removing vclock and advance_vclock()
   RR doesn't give a virtual runtime, maintaining it would be useless.

5. Force_resched only when idle
   Unlike EEVDF, RR gives it fairness by having every task the same full predetermined time slice. Rescheduling mid task is going to reduce the time given.
   So it's only reasonable to reschedule only if we are doing nothing.
   src/mod.rs line 252

I created a file named 'sched.rs' inside usertest to compare the waiting time and completion time for both EEVDF and RR scheduler.
Waiting time is calculated by how long it wait to run - how long it's in the queue
Completion time is calculated by time it finished - how long it's in the queue

For EEVDF

```
--- Scheduler Benchmark Results ---
Task     Wait (ms)    Completion (ms)
----------------------------------------
0        1            110
1        10           57
2        13           130
3        18           69
4        21           92
----------------------------------------
AVG      12           91
------------------------------------
```

For RR

```
--- Scheduler Benchmark Results ---
Task     Wait (ms)    Completion (ms)
----------------------------------------
0        12           117
1        14           63
2        13           123
3        18           65
4        0            84
----------------------------------------
AVG      11           90
------------------------------------
```

EEVDF shows higher variance of waiting time than RR. This is because it prioritizes the task with earlier vdeadlines. RR waiting time is overall nearly identical throughout all 5 tasks. Except task 4 which is because the kernel runs with 4 cores. In conclusion, RR is simpler and fairer. But if you want greater response time for an earlier vdeadline, EEVDF is greater.

```
(base) gluyspt@Gluys-Macbook-Air moss-kernel-modified % just run
[    0.046740] arch::arm64::boot::secondary: CPU 2 online.
[    0.047300] drivers::interrupts::arm_gic_v3: GICv3: Redistributor for core 3 (MPIDR=0x80000003) is awake.
[    0.047656] drivers::interrupts::arm_gic_v3: GICv3: CPU interface for core 3 enabled.
[    0.048000] arch::arm64::boot::secondary: CPU 3 online.
/ # /bin/usertest
Running userspace tests ...
scheduler::benchmark ...
--- Scheduler Benchmark Results ---
Task     Wait (ms)    Completion (ms)
----------------------------------------
0        0            110
1        7            54
2        10           122
3        14           60
4        19           89
----------------------------------------
AVG      10           87
------------------------------------

 OK
usertest::futex::test_futex_bitset ... OK
usertest::futex::test_futex ... OK
usertest::signals::test_interruptible_read_pipe ... OK
usertest::signals::rust_stack_overflow ... OK
usertest::signals::test_segfault_write ... OK
usertest::signals::test_segfault_read ... OK
usertest::signals::test_interruptible_waitpid ... OK
usertest::signals::test_interruptible_nanosleep ... OK
usertest::fs::test_rust_dir ... OK
usertest::fs::test_rust_file ... OK
usertest::fs::test_statx ... OK
usertest::fs::test_utimens ... OK
usertest::fs::test_ftruncate ... OK
usertest::fs::test_truncate ... OK
usertest::fs::test_rename ... OK
usertest::fs::test_symlink ... OK
usertest::fs::test_readdir ... OK
usertest::fs::test_link ... OK
usertest::fs::test_write ... OK
usertest::fs::test_read ... OK
usertest::fs::test_fchown ... OK
usertest::fs::test_chown ... OK
usertest::fs::test_fchmod ... OK
usertest::fs::test_chmod ... OK
usertest::fs::test_chroot ... OK
usertest::fs::test_fchdir ... OK
usertest::fs::test_chdir ... OK
usertest::fs::test_opendir ... OK
usertest::test_clock_sleep ... OK
usertest::test_mincore ... OK
usertest::test_thread_with_name ... OK
usertest::test_parking_lot_mutex_timeout ... OK
usertest::test_rust_mutex ... OK
usertest::test_rust_thread ... OK
usertest::test_fork ... OK
usertest::test_sync ... OK
All tests passed in 4977 ms
/ # exit
```

Level 3:

In src/arch/arm64/exceptions/syscall.rs

I first have a LAST_SYSCALL_NR, a static item to store the syscall number after each call. Then I have prev_syscall = call LAST_SYSCALL_NR.load at the top of handle_syscall function to load the current syscall number before overwriting it.

To handle dmesg calls and print the previous syscall number,
I created a 0x74 match with log::info!("{}", prev_syscall); to show the previous syscall number

```
(base) gluyspt@Gluys-Macbook-Air moss-kernel-modified % just run
[    0.000000] arch::arm64::boot::memory: Reserving FDT Px68000000 (0x100000 bytes)
[    0.000000] arch::arm64::boot::memory: Reserving initrd 0x48000000 - 0x68000000
[    0.000000] libkernel::memory::allocators::phys: Buddy allocator initialized. Managing 524288 pages, 379904 free.
[    0.000000] drivers::fs::dev: devfs initialized
[    0.000000] drivers::interrupts::arm_gic_v3: ARM GICv3 found: dist @ <0x8000000, 0x10000>, rdist @ <0x80a0000, 0xf60000> (stride=0x20000)
[    0.000000] drivers::interrupts::arm_gic_v3: GICv3: Redistributor for core 0 (MPIDR=0x80000000) is awake.
[    0.000000] drivers::interrupts::arm_gic_v3: GICv3: CPU interface for core 0 enabled.
[    0.000000] interrupts: Setting device intc@8000000 as the interrupt root
[    0.000000] interrupts: Device timer claimed interrupt: Ppi(14)
[    0.003000] interrupts: Device pl011@9000000 claimed interrupt: Spi(1)
[    0.008120] libkernel::sync::per_cpu: Setup 20 per_cpu objects.
[    0.009882] interrupts: Device CPU Messenger claimed interrupt: Ipi(0)
[    0.010212] arch::arm64::proc::vdso: VDSO mapped to: 0xffff810000000000 (0x1000 bytes)
[    0.033516] drivers::fs::proc: procfs initialized
[    0.034264] drivers::fs::sys: sysfs initialized
[    0.047066] drivers::interrupts::arm_gic_v3: GICv3: Redistributor for core 1 (MPIDR=0x80000001) is awake.
[    0.047798] drivers::interrupts::arm_gic_v3: GICv3: CPU interface for core 1 enabled.
[    0.048780] arch::arm64::boot::secondary: CPU 1 online.
[    0.049850] drivers::interrupts::arm_gic_v3: GICv3: Redistributor for core 2 (MPIDR=0x80000002) is awake.
[    0.050748] drivers::interrupts::arm_gic_v3: GICv3: CPU interface for core 2 enabled.
[    0.051280] arch::arm64::boot::secondary: CPU 2 online.
[    0.052052] drivers::interrupts::arm_gic_v3: GICv3: Redistributor for core 3 (MPIDR=0x80000003) is awake.
[    0.052934] drivers::interrupts::arm_gic_v3: GICv3: CPU interface for core 3 enabled.
[    0.053552] arch::arm64::boot::secondary: CPU 3 online.
/ # dmesg
[    2.2336358] arch::arm64::exceptions::syscall: 174
[    2.2339070] arch::arm64::exceptions::syscall: 222
/ #
```

Level 4:

In src/arch/arm64/exceptions/syscall.rs

1. Create a ring buffer
   const SYSCALL_HISTORY_SIZE: usize = 64;
   static SYSCALL_HISTORY: SpinLock<VecDeque<u32>> = SpinLock::new(VecDeque::new());

   Use VecDeque to store the history and spinlock to ensure thread safety across multiple CPU cores. I use VecDeque because history has a limit. I use the push_back function to add new history before doing the match case. And pop_front whenever the history is bigger than the SYSCALL_HISTORY_SIZE. Then print the history out when the user calls “dmesg”.

```
(base) gluyspt@Gluys-Macbook-Air moss-kernel-modified % just run
[    2.2547504] arch::arm64::exceptions::syscall:   [7] 73
[    2.2547604] arch::arm64::exceptions::syscall:   [8] 63
[    2.2547698] arch::arm64::exceptions::syscall:   [9] 73
[    2.2547788] arch::arm64::exceptions::syscall:   [10] 63
[    2.2547888] arch::arm64::exceptions::syscall:   [11] 73
[    2.2547982] arch::arm64::exceptions::syscall:   [12] 63
[    2.2548082] arch::arm64::exceptions::syscall:   [13] 66
[    2.2548236] arch::arm64::exceptions::syscall:   [14] 73
[    2.2548370] arch::arm64::exceptions::syscall:   [15] 63
[    2.2548460] arch::arm64::exceptions::syscall:   [16] 66
[    2.2548556] arch::arm64::exceptions::syscall:   [17] 73
[    2.2548644] arch::arm64::exceptions::syscall:   [18] 63
[    2.2548730] arch::arm64::exceptions::syscall:   [19] 66
[    2.2548816] arch::arm64::exceptions::syscall:   [20] 73
[    2.2548912] arch::arm64::exceptions::syscall:   [21] 63
[    2.2549004] arch::arm64::exceptions::syscall:   [22] 66
[    2.2549278] arch::arm64::exceptions::syscall:   [23] 73
[    2.2549726] arch::arm64::exceptions::syscall:   [24] 63
[    2.2549962] arch::arm64::exceptions::syscall:   [25] 66
[    2.2550162] arch::arm64::exceptions::syscall:   [26] 73
[    2.2550288] arch::arm64::exceptions::syscall:   [27] 63
[    2.2550408] arch::arm64::exceptions::syscall:   [28] 64
[    2.2550516] arch::arm64::exceptions::syscall:   [29] 215
[    2.2550618] arch::arm64::exceptions::syscall:   [30] 29
[    2.2550750] arch::arm64::exceptions::syscall:   [31] 134
[    2.2551034] arch::arm64::exceptions::syscall:   [32] 79
[    2.2551188] arch::arm64::exceptions::syscall:   [33] 79
[    2.2551350] arch::arm64::exceptions::syscall:   [34] 79
[    2.2551494] arch::arm64::exceptions::syscall:   [35] 135
[    2.2551598] arch::arm64::exceptions::syscall:   [36] 135
[    2.2551778] arch::arm64::exceptions::syscall:   [37] 220
[    2.2551876] arch::arm64::exceptions::syscall:   [38] 135
[    2.2551966] arch::arm64::exceptions::syscall:   [39] 135
[    2.2552056] arch::arm64::exceptions::syscall:   [40] 154
[    2.2552146] arch::arm64::exceptions::syscall:   [41] 96
[    2.2552236] arch::arm64::exceptions::syscall:   [42] 135
[    2.2552326] arch::arm64::exceptions::syscall:   [43] 135
[    2.2552416] arch::arm64::exceptions::syscall:   [44] 172
[    2.2552504] arch::arm64::exceptions::syscall:   [45] 154
[    2.2552602] arch::arm64::exceptions::syscall:   [46] 29
[    2.2552696] arch::arm64::exceptions::syscall:   [47] 134
[    2.2552792] arch::arm64::exceptions::syscall:   [48] 260
[    2.2552882] arch::arm64::exceptions::syscall:   [49] 134
[    2.2552970] arch::arm64::exceptions::syscall:   [50] 134
[    2.2553060] arch::arm64::exceptions::syscall:   [51] 134
[    2.2553146] arch::arm64::exceptions::syscall:   [52] 134
[    2.2553312] arch::arm64::exceptions::syscall:   [53] 221
[    2.2553408] arch::arm64::exceptions::syscall:   [54] 96
[    2.2553494] arch::arm64::exceptions::syscall:   [55] 214
[    2.2553582] arch::arm64::exceptions::syscall:   [56] 214
[    2.2553670] arch::arm64::exceptions::syscall:   [57] 222
[    2.2553758] arch::arm64::exceptions::syscall:   [58] 226
[    2.2553846] arch::arm64::exceptions::syscall:   [59] 226
[    2.2553934] arch::arm64::exceptions::syscall:   [60] 174
[    2.2554022] arch::arm64::exceptions::syscall:   [61] 116
[    2.2554108] arch::arm64::exceptions::syscall:   [62] 222
[    2.2554198] arch::arm64::exceptions::syscall:   [63] 116
/ #
```